Experiment 8 — Personal Knowledge Base using RAG

Build a Personal RAG-Based Question Answering System | Anson Dias

Objective

Design and implement a Personal Retrieval-Augmented Generation (RAG) question-answering system that retrieves information from an individual's personal document collection using embeddings, a vector database, and a Large Language Model, answering **only** from the uploaded documents and clearly indicating when information is unavailable.

Tech stack

Layer	Choice
Language	Python 3
Framework	LangChain
Document parsing	pypdf (PDF), python-docx (DOCX), built-in (TXT/MD)
Text splitting	RecursiveCharacterTextSplitter
Embeddings	all-MiniLM-L6-v2 (384-dim ONNX, CPU)
Vector database	Chroma (persistent, cosine)
Retrieval	Hybrid: vector similarity + keyword
LLM (generation)	Pluggable: Ollama (free, local) / Groq / Gemini / Claude
Offline fallback	Built-in extractive responder (no key/network)

How it works

Ingestion: the personal documents (resume, academic record, internship, certifications, project report, achievements, SOP — in PDF, DOCX, TXT and Markdown) are loaded, split into overlapping chunks, embedded with all-MiniLM-L6-v2, and stored in a persistent Chroma collection. **Query time:** the question is embedded, the most relevant chunks are retrieved by a hybrid vector + keyword search, and an LLM (or the offline extractive responder) writes an answer grounded strictly in those chunks. If no retrieved passage matches, the system replies that the information is not in the knowledge base instead of guessing.

Knowledge base documents

- **resume.md** — contact, education, skills, projects, experience
- **academic_transcript.pdf** — university, programme, GPA, coursework
- **internship_certificate.docx** — internship experience (CostumePeti)
- **certifications.txt** — certificates and courses
- **project_report.md** — project report (accounting automation)
- **achievements_and_awards.md** — key achievements & highlights
- **statement_of_purpose.docx** — SOP with interests + contact

Walkthrough (screenshots)

Step 1 — Generate the Student Knowledge Base documents

```
ansondias@rag:~/experiment-8-personal-rag$ python build_knowledge_base.py
Wrote 7 documents to 'Student Knowledge Base/':
  - academic_transcript.pdf          (2,718 bytes)
  - achievements_and_awards.md       (989 bytes)
  - certifications.txt               (484 bytes)
  - internship_certificate.docx      (36,989 bytes)
  - project_report.md                (964 bytes)
  - resume.md                        (4,351 bytes)
  - statement_of_purpose.docx        (37,177 bytes)
```

Step 1 — Generate the Student Knowledge Base documents.

Step 2 — Personal documents (PDF / DOCX / TXT / MD)

```
ansondias@rag:~/experiment-8-personal-rag$ ls -lh "Student Knowledge Base"
total 104K
-rw-r--r-- 1 root root 2.7K Oct 12 21:44 academic_transcript.pdf
-rw-r--r-- 1 root root  989 Oct 12 21:44 achievements_and_awards.md
-rw-r--r-- 1 root root  484 Oct 12 21:44 certifications.txt
-rw-r--r-- 1 root root  37K Oct 12 21:44 internship_certificate.docx
-rw-r--r-- 1 root root  964 Oct 12 21:44 project_report.md
-rw-r--r-- 1 root root 4.3K Oct 12 21:44 resume.md
-rw-r--r-- 1 root root  37K Oct 12 21:44 statement_of_purpose.docx
```

Step 2 — The personal documents across PDF, DOCX, TXT and Markdown.

Step 3 — Ingest: load -> split -> embed -> store in Chroma

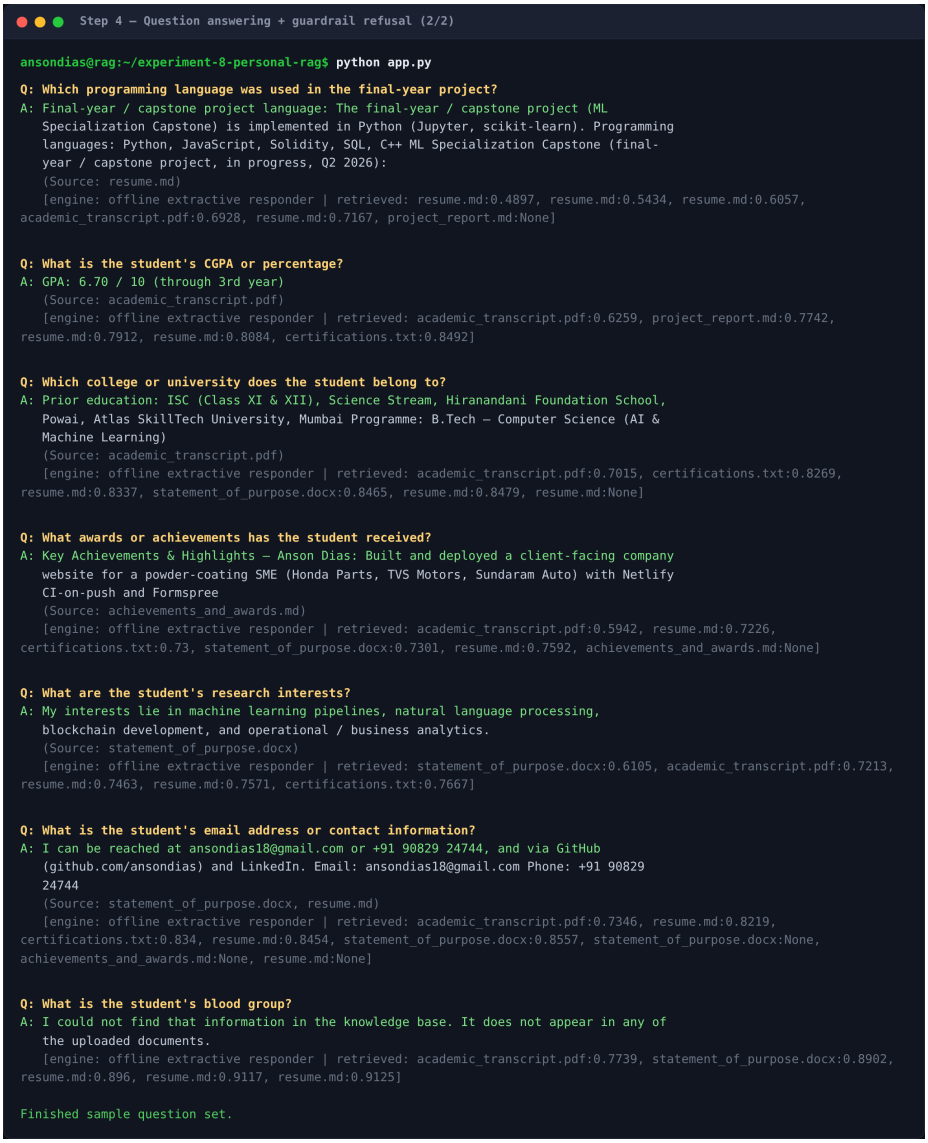
```
ansondias@rag:~/experiment-8-personal-rag$ python ingest.py
[1/4] Loading documents from 'Student Knowledge Base/'
  loaded  academic_transcript.pdf          (710 chars)
  loaded  achievements_and_awards.md       (986 chars)
  loaded  certifications.txt               (477 chars)
  loaded  internship_certificate.docx      (655 chars)
  loaded  project_report.md                (961 chars)
  loaded  resume.md                        (4,322 chars)
  loaded  statement_of_purpose.docx        (961 chars)
[2/4] Splitting 7 documents into chunks (size=750, overlap=120)
      -> 17 chunks
[3/4] Embedding chunks with all-MiniLM-L6-v2 (384-dim ONNX, CPU)
[4/4] Storing vectors in Chroma at 'chroma_db/'

Done. Indexed 17 chunks from 7 documents.
```

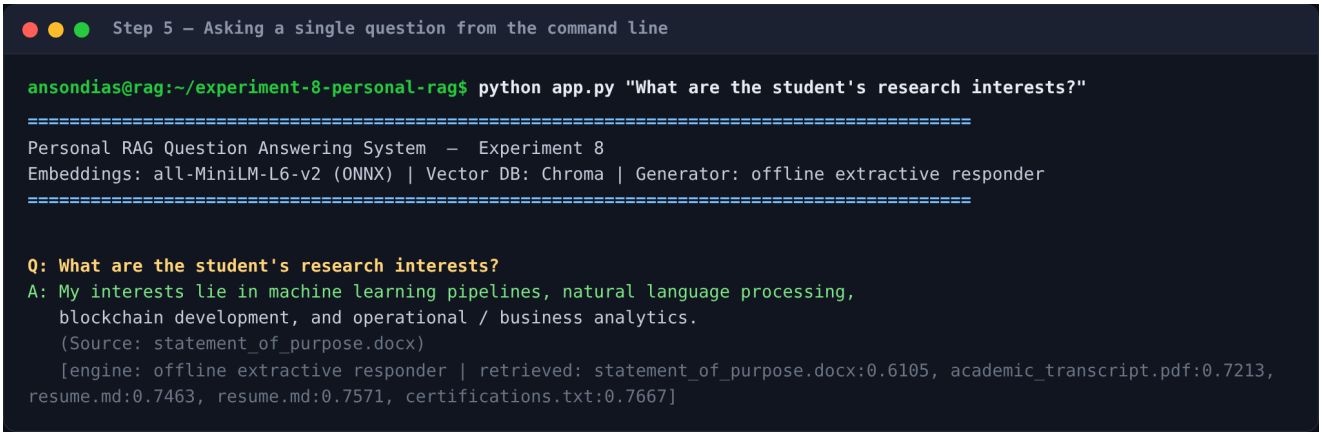
Step 3 — Ingestion: load → split → embed → store in Chroma.



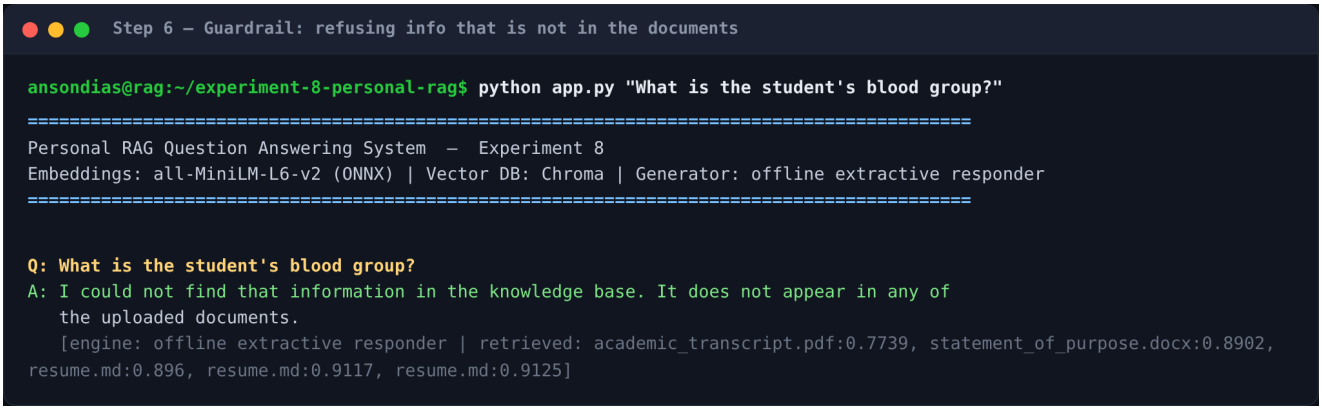
Step 4 — Question answering over the knowledge base (1/2).



Step 4 — Question answering + guardrail refusal (2/2).



Step 5 — Asking a single question from the command line.



Step 6 — Guardrail: refusing information not in the documents.

Results — all required questions answered

Every question required by the assignment is answered from the documents; a question whose answer is absent (blood group) is correctly refused.

Question	Answered (from documents)
Full name	Anson Dias
Programming languages	Python, JavaScript, Solidity, SQL, C++
Internship	AI Content & Prompt Engineering Intern, CostumePeti (2024)
Certifications	Google ML Crash Course; DeepLearning.AI ML Specialization
Final-year project language	Python (ML Specialization capstone)
CGPA	6.70 / 10 (through 3rd year)
College / university	Atlas SkillTech University, Mumbai
Research interests	ML pipelines, NLP, blockchain, operational analytics
Email / contact	ansondias18@gmail.com, +91 90829 24744
Blood group	Not in knowledge base → refused

Conclusion

The system builds a searchable vector knowledge base from personal documents and answers natural-language questions grounded only in retrieved content, refusing anything not present. The answer generator is pluggable across free LLMs (Ollama runs locally with no API key); the retrieval pipeline and the grounding guardrail are identical regardless of generator.